

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

Курсовая работа

по дисциплине: «Машинное обучение»

на тему: «Атаки в системах машинного обучения»

Студент М-РИТ-25-1 _____ Станиславчук С.М.

Руководитель

Доцент, д.т.н. _____ Сараев П.В.

Липецк, 2025

Содержание

1. Цель работы 3

2. Задание 3

3. Исходный датасет 4

4. Программа для обучения модели 5

5. Анализ уязвимостей модели 7

6. Атакующие алгоритмы 9

7. Выводы 14

1. Цель работы

Реализовать и проанализировать модель машинного обучения, решающую задачу классификации, а также исследовать её устойчивость к различным видам атак.

2. Задание

Выбранная тематика: 8. Атаки в системах машинного обучения.

1. Подобрать интересующее соревнование и извлечь из него датасет

2. Написать программу, которая обучит модель на выбранном датасете

3. Провести анализ уязвимости полученной модели

4. Провести атаки на модель.

5. Сравнить атакованные и нетронутые результаты.

6. Предложить усиление слабых мест модели.

3. Исходный датасет

В качестве задания, был выбран датасет, использующийся в соревновании «Titanic - Machine Learning from Disaster» с сервиса ***kaggle.com***, тематикой которого, как видно из названия, является крушение одноименного судна.

Датасет включает в себя всех пассажиров "Титаника", находящихся на нем в момент крушения. Задача соревнования: обучить модель таким образом, чтобы она могла определить по входным данным пассажира спасется он или нет в результате крушения.

1. `survival` — факт выживания (0 — нет, 1 — да)
2. `pclass` — класс билета (1 — первый, 2 — второй, 3 — третий)
3. `sex` — пол пассажира
4. `age` — возраст
5. `sibsp` — количество супругов и детей на борту
6. `parch` — количество родителей и детей на борту
7. `ticket` — номер билета
8. `fare` — стоимость проезда
9. `cabin` — номер каюты
10. `embarked` — порт посадки (C — Шербур, Q — Квинстаун, S — Саутгемптон)

Пояснение к переменным:

pclass: показатель социально-экономического статуса

- 1-й - Высший
- 2-й - Средний
- 3-й - Нижний

age: возраст указывается дробно, если он меньше 1. sibsp: набор данных определеяет семейные отношения следующим образом: Дети - брат, сестра, сводный брат, сводная сестра. Супруг - муж, жена

parch: Набор данных определяет семейные отношения следующим образом: Родитель - мать, отец Ребенок - дочь, сын, падчерица, пасынок (Некоторые дети путешествовали только с няней, для них parch = 0)

Датасет состоит из двух частей: train и test, оба в формате .csv. Файл train.csv состоит из 418 записей (пассажиров), test.csv из 891.

4. Программа для обучения модели

Так как модель следует обучить предугадыванию выживания, то возможны лишь два исхода для пассажира: выживет или нет. Правильным решением для такой задачи будет использование методов классификации для обучения модели. Всего будет 2 класса: выживший (1) и невыживший (0). Таким образом, задача является бинарной классификацией.

Для обучения модели была реализована программа на языке Python с использованием библиотек `pandas`, `scikit-learn`, `numpy`, `joblib`.

- `pandas` — для загрузки датасета, обработки пропущенных значений и преобразования признаков;
- `numpy` — для выполнения численных операций и работы с массивами данных;

- `scikit-learn` — для реализации алгоритмов машинного обучения, разбиения данных на обучающую и тестовую выборки, а также оценки качества модели;
- `joblib` — для сохранения и последующей загрузки обученной модели.

На первом этапе выполняется загрузка данных и их предварительная обработка. Из датасета удаляются признаки, не оказывающие существенного влияния на результат классификации (идентификаторы пассажиров, имена и номера билетов). Пропущенные значения возраста заменяются медианным значением, а отсутствующие данные о порте посадки — наиболее часто встречающимся значением.

Категориальные признаки, такие как пол пассажира и порт посадки, преобразуются в числовой формат с использованием бинарного кодирования. Информация о каюте дополнительно обрабатывается: из строкового значения выделяется палуба и числовой номер каюты, что позволяет сохранить полезные пространственные характеристики расположения пассажиров на судне.

В качестве модели классификации был выбран алгоритм Random Forest, основанный на ансамбле решающих деревьев. Данный алгоритм обладает устойчивостью к выбросам, не требует масштабирования признаков и способен эффективно обрабатывать нелинейные зависимости между входными данными.

Обучение модели производится на обучающей выборке с последующим разделением данных на тренировочную и валидационную части. Качество классификации оценивается с использованием метрик точности (Accuracy) и площади под ROC-кривой (AUC). Результаты выполнения

программы представлены на рисунке 1.

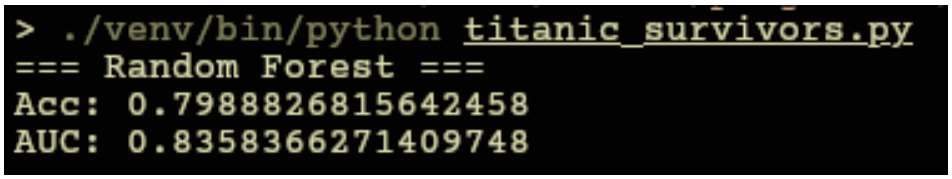


Рисунок 1 — Результат выполнения обученной модели

Как видно из рисунка, показатель точности достаточно высокий ≈ 0.7989 Таким образом, определение класса для большинства объектов выборки прошло успешно. А показатель AUC еще выше ≈ 0.8358, что свидетельствует о высокой способности модели различать пассажиров, выживших и погибших в результате крушения. Как раз та самая точность, на которой будут заметны результаты атак.

5. Анализ уязвимостей модели

Несмотря на достаточно высокие значения метрик качества, модель машинного обучения может обладать рядом уязвимостей, связанных с особенностями обучающего датасета, предобработки данных и используемого алгоритма.

Под уязвимостью в данном случае понимается возможность искусственного изменения входных данных таким образом, чтобы модель выдала неверный результат, либо снижение устойчивости предсказаний при незначительных изменениях признаков.

Зависимость от ключевых признаков.

График самых влиятельных на результат признаков представлен на рисунке 2.

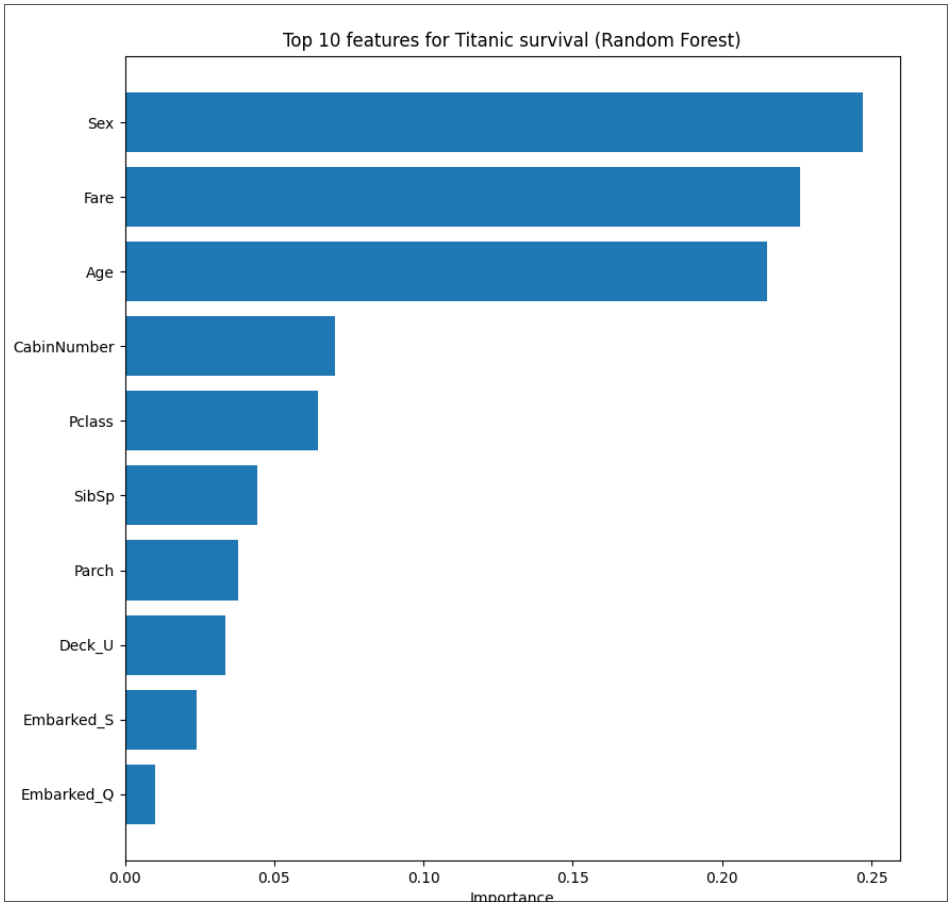


Рисунок 2 — График с самыми влиятельными признаками

В данной модели наибольшее влияние на результат классификации оказывают признаки `Sex`, `Fare` и `Age` с влиянием >= 0.2. `CabinNumber`, `Pclass` и остальные признаки имеют не такое большое влияние на результат <= 0.07.

Это означает, что модель сильно полагается на ограниченное количество признаков.

Отсутствие гладкости предсказаний.

Random Forest формирует решение на основе набора пороговых условий, что приводит к скачкообразному изменению результата при небольших изменениях входных данных. Уязвимостью может быть даже незначительное изменение одного признака (например, возраста на 1-2 года) может изменить путь прохождения по деревьям и привести к

противоположному результату классификации.

6. Атакующие алгоритмы

Теперь, когда уязвимости известны, нужно выбрать атакующие алгоритмы.

Начну с того, зачем вообще нужно проводить атаку на модель.

Системы машинного обучения в своей основе являются строго зависимыми от данных, что влечёт за собой ряд существенных последствий. Любые изменения во входных данных неизбежно отражаются на поведении модели. При этом обучение моделей машинного обучения осуществляется лишь на ограниченном подмножестве данных — тренировочном наборе. На этапе эксплуатации модель начинает взаимодействовать с реальными данными, представляющими генеральную совокупность. Эти данные по своим статистическим характеристикам могут в целом отличаться от тех, которые использовались при обучении и тестировании модели. График аппроксимации функции при наличии подменных данных представлен на рисунке 3.

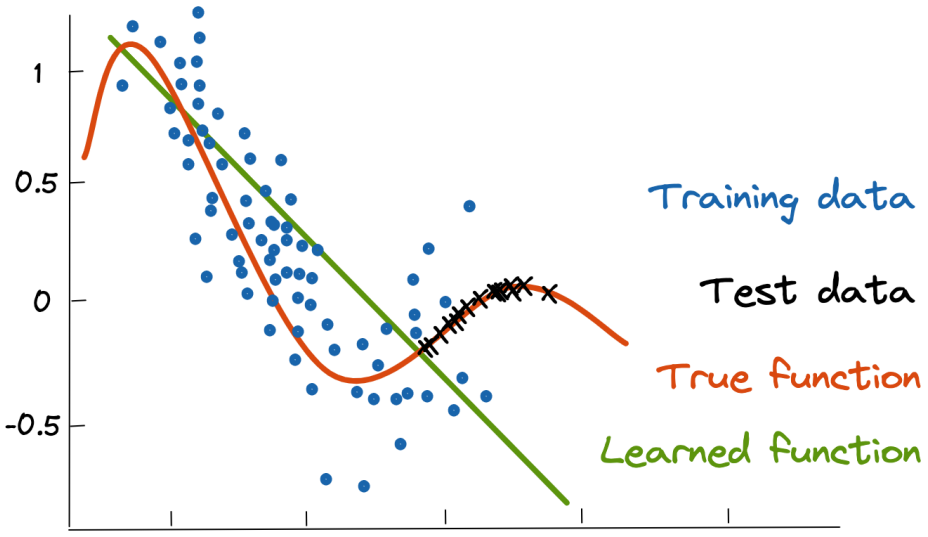


Рисунок 3 — График аппроксимации функции

Модель стремится аппроксимировать неизвестную зависимость, изображённую на графике красной линией. Синим цветом на рисунке обозначены данные тренировочного набора.

Данные тренировочного набора хорошо обобщаются моделью, и аппроксимирующая зелёная прямая демонстрирует высокие значения метрик до тех пор, пока реальные данные сохраняют сходные характеристики с обучающим набором. Однако при изменении свойств входных данных величина потерь начинает возрастать. Таким образом, изменение распределения данных напрямую приводит к изменению поведения модели. При этом такие изменённые данные остаются полностью легитимными для модели и не выходят за рамки допустимого входного пространства.

Следовательно, в процессе эксплуатации модель машинного обучения может демонстрировать значения метрик, существенно отличающиеся от достигнутых на этапе тестирования. На рисунке 3 показан наиболее простой вид расхождения между реальными и тренировочными данными — ковариантный сдвиг. Вместе с тем наибольшую опасность для корректной работы модели представляет так называемый сдвиг концепции, заключающийся в изменении зависимости между входными и выходными переменными.

Дополнительно возникает вопрос о наличии противоборствующей стороны, которая целенаправленно подбирает такие входные данные, способные негативно повлиять на работу модели. Разница между аппроксимирующей прямой и истинной функцией на рисунке 3 соответствует величине потерь. Поведение функции потерь стандартным образом определяется её производной по входным данным. Для многомерного пространства признаков такими характеристиками являются частные производные, а их совокупность образует якобиан. На рисунке 3 пред-

ставлена иллюстрация атак уклонения, основанных на целенаправленных модификациях входных данных, которые приводят к изменению результата классификации после добавления к корректно распознаваемому изображению шума, вычисленного с использованием якобиана функции потерь.

Следует отметить, что подобные модификации входных данных тесно связаны с проблемой устойчивости моделей машинного обучения. Как правило, алгоритм, в котором ошибка, присутствующая во входных данных или возникающая в процессе вычислений, не возрастает или увеличивается незначительно на каждом шаге, считается устойчивым. В противном случае, когда погрешность существенно накапливается от шага к шагу, алгоритм классифицируется как неустойчивый. Таким образом, устойчивость алгоритма представляет собой количественную меру его чувствительности к изменениям исходных данных. На рисунке 4 приведён пример того, как незначительное изменение входных данных может приводить к существенному снижению результативности модели.

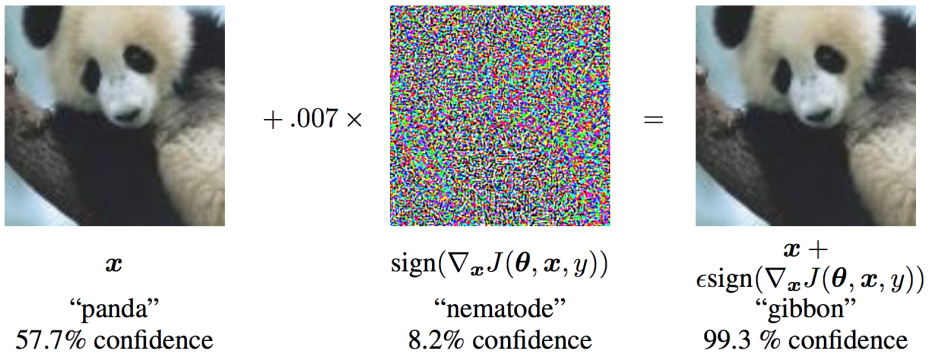


Рисунок 4 — Враждебная атака с применением шума

Вот только есть один нюанс. Как упоминалось ранее, модель использует алгоритм Random Forest. Выражение для предсказания одного дерева в ансамбле Random Forest представлено в формуле (1).

$$f_{\mathrm{DT}}(x) = \sum_k c_k * 1_{x \in R_k} \tag{1}$$

где: $R_k \subset \mathbb{R}^d$ — области признакового пространства, соответствующие листьям дерева,

$c_k \in [0, 1]$ — значение предсказания в листе,

$1_{x \in R_k}$ — индикаторная функция, показывающая, что x попадает в лист R_k.

Данная функция недифференцируема. Для иллюстрации была использована выборка случайных чисел с дополнительным ограничением $y = x > 30$. График функции приведён на рисунке 5.

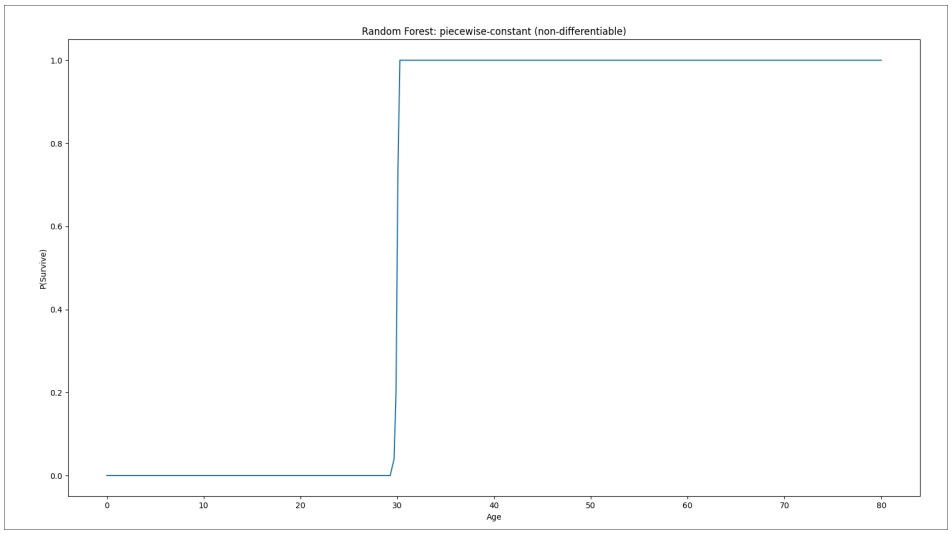


Рисунок 5 — Предсказание одного дерева Random Forest для $x > 30$

Поскольку модели на основе деревьев решений недифференцируемы, прямое вычисление якобиана функции потерь для них невозможно. В связи с этим используется surrogate-модель на основе нейронной сети, аппроксимирующая границу решений исходного классификатора. В отличие от Random Forest, нейронная сеть задаёт гладкую и дифференцируемую функцию, что позволяет вычислять производные функции потерь по

входным признакам и анализировать чувствительность модели к малым возмущениям входных данных.

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(f_{\text{RF}}(x_i) - \hat{f}_{\text{NN}}(x_i; \theta) \right)^2 \tag{2}$$

где: $f_{\text{RF}}(x)$ — предсказание Random Forest,

$\hat{f}_{\text{NN}}(x; \theta)$ — surrogate нейросеть с параметрами θ,

$\hat{f}_{\text{NN}}(x; \theta)$ — градиент surrogate, используемый для оптимизации,

График функции surrogate-модели нейронной сети представлен на рисунке 6.

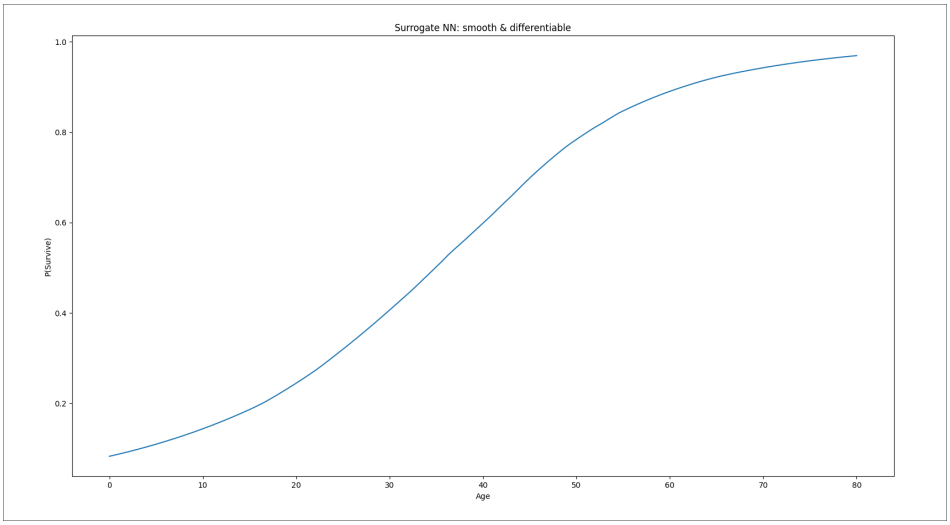


Рисунок 6 — Предсказание surrogate-модели на основе нейронной сети

Такое поведение функции можно объяснить тем, что Surrogate NN использует функцию активации. Функции активации привносят нелинейность в сеть, позволяя ей моделировать сложные структуры данных. конкретно в моем примере, это функция $ReLU$ (3). А в качестве функции затухания используется $Sigmoid$ (4)

$$ReLU(x) = max(0, x) \tag{3}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{4}$$

Теперь для выявления характеристик, чувствительность которых основана на якобиане, напишу программу (см. Приложение 3). Результатом данной программы будет список характеристики с высокой чувствительностью по якобиану для рассматриваемой модели Random Forest. Выполненная программа представлена на рисунке 7.

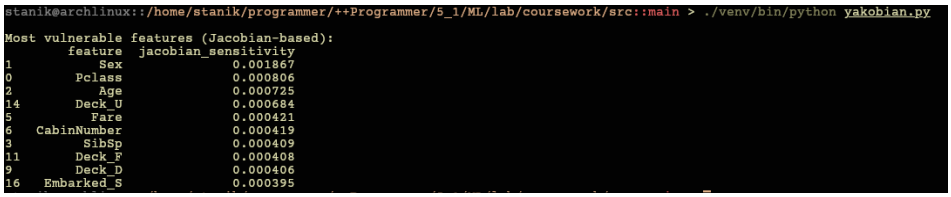


Рисунок 7 — Характеристики с высокой чувствительностью по якобиану

Для атаки буду использовать алгоритм Surrogate NN. Её цель приближённо учиться копировать поведение уже обученной модели, конкретно - границу решений: где "выжил"и где "умер". В ходе суррогатного моделирования мы строим подменную (или суррогатную) модель методами машинного обучения для достаточно точной аппроксимации результатов моделирования. Впоследствии эта обученная статистическая модель может быть использована для замены исходной компьютерной симуляции.

$$\delta Loss/\delta X$$

В программе я использую 60 эпох

7. Выводы

Приложение 1. Код программы

```
import os
import joblib

from sklearn.metrics import accuracy_score, roc_auc_score

from sklearn.ensemble import RandomForestClassifier

from data_preprocessing import get_processed_data

# ----------------------------------------------------------
# Prepare output dir
# ----------------------------------------------------------
os.makedirs("models", exist_ok=True)

# ----------------------------------------------------------
# Load data
# ----------------------------------------------------------
X_train, X_val, y_train, y_val, df, scaler = get_processed_data(
    data_dir="titanic/", test_size=0.2, random_state=42
)

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)

# Save scaler
```

```
joblib.dump(scaler, "models/scaler.pkl")


# ============================================================
# Random Forest
# ============================================================
rf = RandomForestClassifier(n_estimators=400, random_state=42)
rf.fit(X_train, y_train)

pred_rf = rf.predict(X_val)
proba_rf = rf.predict_proba(X_val)[:, 1]

print("=== Random Forest ===")
print("Acc:", accuracy_score(y_val, pred_rf))
print("AUC:", roc_auc_score(y_val, proba_rf))
print()

joblib.dump(rf, "models/random_forest.pkl")
```